MATH4515 Statistical and Computational Methods in Financial Mathematics

Lecture Note 2

Monte Carlo Simulation and Bootstrapping method

Introduction

In studying many real life problems in finance and risk management, it always involves the computation of various expected values and probabilities:

- When we price financial derivatives (e.g. options) in a market, the current price of any derivative, denoted by V_0, can be expressed as
$$V_0 = e^{-rT}\mathbb{E}^Q[V_T(S_T)],$$
where S_T denotes the asset price at time T and $V_T(S_T)$ denotes the payoff of the derivative at time T (which depends on S_T).
- An investor has constructed a portfolio for investment. He/She wishes to study the performance of the portfolio. To do so, one needs to check the probability distribution of the portfolio return r_P (percentage return/logreturn), i.e. $P(r_P > c)$.
- When an insurance company manages the risk in a portfolio of insurance policies, it needs to study the distribution of the total claims. That is,
$$S = X_1 + X_2 + \cdots + X_N,$$
where X_i is the loss amount in a claim event and N is claim frequency.

In many cases, it is difficult (or simply infeasible) to find the explicit solution for those problems.

Monte Carlo simulation is one of the efficient methods in obtaining solutions of these problems. In this Chapter, we shall study

- The basic principle of Monte Carlo simulation
- The application of Monte Carlo simulation in finance and statistics.
- Trouble-Shooting and enhancement in Monte Carlo simulation – Rejection sampling, Important Sampling.

Next, we shall study Bootstrapping method. This method is an extension of Monte Carlo simulation which aims to deals with the case when the true distribution of the target random variable is unknown. This method is very useful in solving some statistical problems such as

- Finding the confidence interval of μ when the population of X is *not* normally distributed.

Monte Carlo Simulation and its application

This method is a computer-based approach for estimating the expectation $\mathbb{E}[X]$ through *random sampling*.

Recall that $\mu = \mathbb{E}[X]$ can denote the *population mean* of a population representing X. Given a random sample $\{X_1, X_2, \dots, X_n\}$ drawn from the population, one can estimate the "population mean" by *sample mean*. That is,

$$\mathbb{E}[X] = \mu \approx \frac{X_1 + X_2 + \cdots + X_n}{n}.$$

By the strong law of large number, this approximation is accurate when the sample size n is sufficiently large. Given the probability distribution of a random variable X, one can estimate $\mathbb{E}[f(X)]$ as follows:

Step 1: Generate a set of random number $\{X_1, X_2, \dots, X_n\}$ from the distribution of X (see *Remark below*)

Step 2: Calculate $f(X_1), f(X_2), \dots, f(X_n)$.

Step 3: Estimate $\mathbb{E}[f(X)]$ by $\frac{f(X_1)+f(X_2)+\cdots+f(X_n)}{n}$.

Remarks

The key step of the Monte Carlo simulation is to calculate a random sample from a given probability distribution of X. There are 2 ways to achieve that.

Method 1: In R/RStudio, one can generate the random numbers from some well-known distributions by the built-in command. For example:

- "**r**norm(n, μ, σ)" for normal distribution,
- "**r**lnorm(n, μ, σ)" for lognormal distribution,
- "**r**exp(n, λ)" for exponential distribution.

Method 2: Alternatively, one can generate a *continuous random variable* from a probability distribution $F_X(x) = P(X \leq x)$ as follows:

- Step 1: Generate a random number from uniform distribution over $[0, 1]$.
- Step 2: Given the random number U obtained, we get the required random number Y by solving

$$F_X(Y) = U \Leftrightarrow Y = F_X^{-1}(U).$$

(*Note: Normally, the above equation has unique solution when X is a continuous random variable. This method is known as inverse transformation method)

One can show that the probability distribution of Y is same as that of X.

Example 1 (Generating Random variable using inverse transformation)

(a) A random variable X has exponential distribution with the probability density function $f_X(x) = \lambda e^{-\lambda x},\ x > 0$. Generate a random sample using inverse transformation method.

(b) A random variable Y has Pareto distribution with probability density function $f_X(x) = \frac{\alpha\theta^{\alpha}}{(x+\theta)^{\alpha+1}},\ x > 0$. Generate a random sample using inverse transformation method.

☐Solution of (a)

Firstly, the cumulative distribution function of X is given by

$$F_X(x) = \int_0^x \lambda e^{-\lambda y} dy = \left[-e^{-\lambda y}\right]_0^x = 1 - e^{-\lambda x}.$$

Given the random number U generated from uniform distribution over $[0,1]$, one can obtain the sample data X_i by

$$F_X(X_i) = U \Rightarrow 1 - e^{-\lambda X_i} = U \Rightarrow X_i = -\frac{1}{\lambda}\ln(1-U).$$

☺Solution of (b)

The corresponding c.d.f. of X is $F_X(x) = \int_0^x \frac{\alpha\theta^\alpha}{(y+\theta)^{\alpha+1}}\,dy = \cdots = 1 - \left(\frac{\theta}{x+\theta}\right)^\alpha$.

So the corresponding formula for obtaining the sample data is

$$F_X(X_i) = 1 - \left(\frac{\theta}{X_i+\theta}\right)^\alpha = U \Rightarrow X_i = \theta\left[\frac{1}{(1-U)^{\frac{1}{\alpha}}} - 1\right].$$

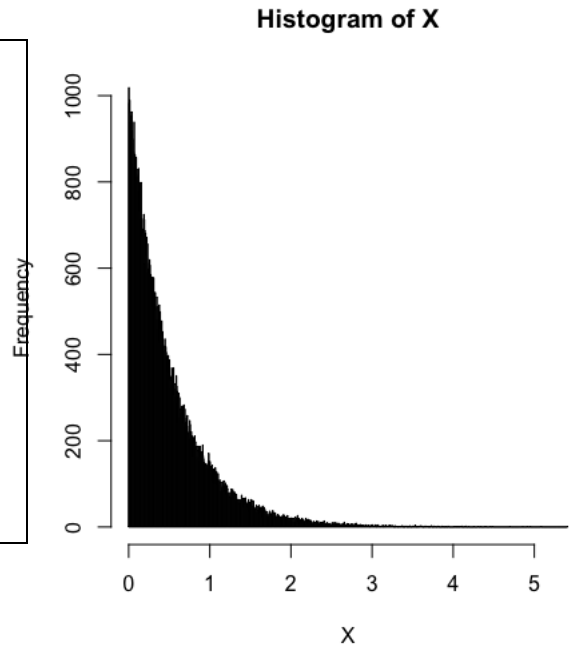
Remarks: One can execute the inverse transformation with the following command:

(For the case of exponential distribution with $\lambda = 2$)

```
n=100000 #Sample size of the data
lambda=2 #Parameter value of the distribution

#Function for generating random number from the distribution
(inverse transformation)
cdfinverse<-function(U,lambda){
  X=-(1/lambda)*log(1-U)
}

U=runif(n,min=0,max=1)
X=cdfinverse(U,lambda)
```



Example 2 (A quick testing on Monte Carlo simulation)

In this example, we would like to examine the effectiveness of Monte Carlo simulation and properties of the estimates.

We consider X be a random variable with lognormal distribution with parameters $\mu = 2$ and $\sigma = 1$. We have $\mathbb{E}[X^2] = e^{2\mu+2\sigma^2} = e^6 = 403.2388$.

Next, we estimate $\mathbb{E}[X^2]$ using Monte Carlo simulation with different values of n (using R), the result is summarized in the following table.

n	$\mathbb{E}[X^2]$ (1st run)	$\mathbb{E}[X^2]$ (2nd run)	$\mathbb{E}[X^3]$ (2nd run)
1000	494.1017	388.8681	527.8169
10000	364.3989	415.3666	398.0703
100000	403.8357	412.044	403.206
1000000	406.2563	403.5084	408.7958
10000000	403.4926	402.5085	403.6652
100000000	403.133	403.3188	403.302

Table 1: Estimates from Monte Carlo simulation

Discussion of results

- We observe from the above table that the estimate obtained is a *random variable*. It is because the estimate is calculated based on the *random sample*.
- The estimate has huge fluctuation when the sample size is small (say $n = 1000$). However, the fluctuation becomes smaller as n increases.
- The estimate converges to the true value (i.e. 403.2388) when n is very large (e.g. $n = 100000000$). It confirms the theoretic result established in earlier section.

Measuring Estimation Error – Standard Error and Percentage Error

It is often the case that the true value is unknown in practice. To estimate the estimation error, we define the standard error (denoted by SE) as

$$SE = \sqrt{\mathbb{E}[(\bar{X} - \mu)^2]}$$

Using the fact that $\mathbb{E}[\bar{X}] = \mathbb{E}[X] = \mu$, we can deduce that

$$SE = \sqrt{\mathbb{E}[(\bar{X} - \mathbb{E}[\bar{X}])^2]} = \sqrt{Var(\bar{X})} = \sqrt{\frac{1}{n}Var(X)} \approx \sqrt{\frac{1}{n}\left(\frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2\right)}.$$

On the other hand, we define the percentage error by $PE = \frac{SE}{Estimate} = \frac{SE}{\bar{X}}$.

The following table shows the standard error of the estimation of previous example. The number in the bracket represents the *percentage error*)

n	$\mathbb{E}[X^2]$ (1st run)	$\mathbb{E}[X^2]$ (2nd run)	$\mathbb{E}[X^3]$ (2nd run)
1000	107.8882 (21.8352%)	81.0513 (20.8426%)	147.3066 (27.9087%)
10000	15.279 (4.1929%)	24.8436 (5.9812%)	23.0787 (5.7976%)
100000	7.7279 (1.9136%)	8.6243 (2.0931%)	8.9432 (2.2181%)
1000000	2.6635 (0.6556%)	3.4279 (0.8495%)	3.1792 (0.7777%)
10000000	0.9013 (0.2234%)	0.8518 (0.2116%)	0.9072 (0.2247%)
100000000	0.2950 (0.0732%)	0.3117 (0.0773%)	0.2838 (0.0704%)

We observe that the standard error (or percentage error) of the estimate is getting smaller when $n \to \infty$. When we adopt Monte Carlo simulation in practice, one can choose n such that the percentage error is less than 0.5% (ideally 0.1%).

Example 3 (Application on derivative pricing)

We consider a T-year European call option on a company stock. The strike price of the option is X. So the holder will receive a payoff of $V_T = \max(S_T - X, 0)$ after T years.

On the other hand, the current price of the stock is S_0 and its future price process is governed by the following equation (known as Black-Scholes model):

$$dS_t = \mu S_t dt + \sigma S_t dW_t \Leftrightarrow S_t = S_0 e^{\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma W_t},$$

where W_t has normal distribution with mean 0 and variance t. Here, $\mu \in \mathbb{R}, \sigma > 0$ denotes expected return and volatility of the stock respectively.

According to *risk neutral valuation principle* (or martingale pricing theory), the current price of the call option can be expressed as

$$V_0 = e^{-rT}\mathbb{E}^Q[V_T(S_T)] = e^{-rT}\mathbb{E}^Q[\max(S_T - X, 0)] \dots \dots (*)$$

where Q denotes *risk neutral probability measure* and

$$S_T = S_0 e^{\left(r - \frac{\sigma^2}{2}\right)t + \sigma W_T^Q},$$

where W_T follows normal distribution with mean 0 and variance T.

We take $S_0 = 100, r = 3\%, \sigma = 20\%,\ T = 1$ and $X = 110$. We can estimate the price by applying Monte Carlo simulation on $(*)$:

```
N=100000 #Number of simulation runs
S0=100
X=110
sigma=0.2
r=0.03
T=1
VT=rep(0,N)

for (i in 1:N){
  WT=rnorm(1,mean=0,sd=sqrt(T))
  ST=S0*exp((r-(sigma^2)/2)*T+sigma*WT)
  VT[i]=exp(-r*T)*max(ST-X, 0)}

Estimate=mean(VT) #estimate
SE=sd(VT)/sqrt(N) #Standard error
PE=SE*100/Estimate #Percentage error (in percentage)
```

	Estimate	Standard Error	Percentage Error
$\boldsymbol{N = 10000}$	5.269533	0.1079814	2.049164%
$\boldsymbol{N = 100000}$	5.314647	0.034771417	0.6542574%
$\boldsymbol{N = 1000000}$	5.293702	0.01096328	0.2071004%
$\boldsymbol{N = 10000000}$	5.293250	0.00346183	0.0654008%

(*Using the Black-Scholes formula, the actual price of the call option is about $5.293399)

Example 4 (Application on analysing risk in P&C insurance)

We consider an insurance policy that covers certain type of losses (e.g. travel insurance, motor vehicle insurance). We let

- N denotes the number of claim events/loss events caused by the policyholder throughout the life of the insurance;
- X_i denotes the loss amount (or claim amount) made in i^{th} claim event.
- N and X_i are independent

The total claim amount can be expressed as

$$S = X_1 + X_2 + \cdots + X_N.$$

Suppose that N has Poisson distribution with mean $\lambda = 4$ and X_i has the following probability density function: $f_{X_i}(x) = \frac{3}{x^4},\ x > 1.$

Calculate the following using Monte Carlo simulation

(a) $\mathbb{E}[S]$ (*Note: Theorical Value: $\mathbb{E}[S] = \mathbb{E}[N]\mathbb{E}[X_i] = 6$)

(b) $P(S > 10)$

(c) π_P such that $P(S \leq \pi_P) = 95\%$. (*The number π_P is known as Value-at-risk at 95% level)

□Solution of (a) and (b)

The algorithm can be described as follows:

- Generate N from Poisson distribution by "rpois()"
- Given N, generate $X_1, X_2, \ldots, X_N$ from the distribution of X by inverse transformation. (If $N = 0$, no need to generate X_i and set $S = 0$)
- Calculate $S^{(i)} = X_1 + X_2 + \cdots + X_N$.
- Repeat the process for n times to get enough sample.

For $\mathbb{E}[S]$, we can calculate the number by

$$\mathbb{E}[S] = \frac{S^{(1)} + S^{(2)} + \cdots + S^{(n)}}{n}.$$

For $P(S > 10)$, we calculate the probability by

$$P(S > 10) = \mathbb{E}\left[\mathbf{1}_{\{S>10\}}\right] = \frac{\mathbf{1}_{\{S^{(1)}>10\}} + \cdots + \mathbf{1}_{\{S^{(n)}>10\}}}{n}.$$

The corresponding R-code is given as follows:

```
lambda=4 #True value of parameter
n=100000 #Number of simulation runs

#Generate random variables from X via inverse transformation
cdfinverseX<-function(U){
  X=(1-U)^(-1/3)
  return(X)}

S=rep(n,0)
#Generate sample values of S
for (i in 1:n){
  N<-rpois(1,lambda)
  U<-runif(N,min=0,max=1)
  X<-cdfinverseX(U)
  S[i]=sum(X)
}

#Calculate E[S]
meanestimate=mean(S)
meanSE=sd(S)/sqrt(n)
meanPE=meanSE*100/meanestimate

#Calculate P(S>10)
Y<-ifelse(S>10,1,0)
probestimate=mean(Y)
probSE=sd(Y)/sqrt(n)
probPE=probSE*100/probestimate
```

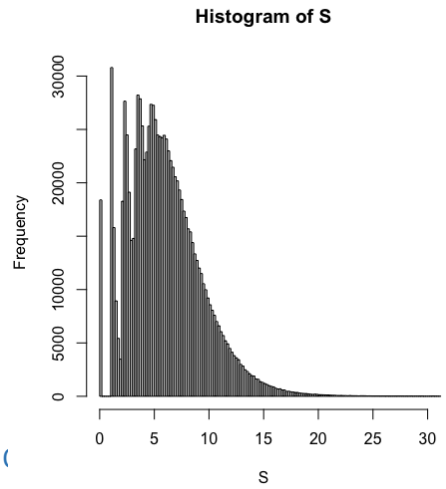
The following table summarizes the simulation results:

Number of simulation runs	Estimate of $\mathbb{E}[S]$	Standard Error (Percentage Error)	Estimate of $P(S > 10)$	Standard Error (Percentage Error)
$n = 10000$	6.006708	0.0349112 (0.5812%)	0.1204	0.0032545 (2.70303%)
$n = 100000$	6.005387	0.0109161 (0.18177%)	0.11793	0.0010199 (0.86485%)
$n = 1000000$	5.990561	0.0034422 (0.05746%)	0.116387	0.00032069 (0.275536%)

☐Solution of (c)

One can compute this by plotting the distribution of S using the sample and determine $VaR_p(S)$ by finding the corresponding percentiles (95% in our case)

```
hist(S,breaks=1000,xlim=c(0,30))
> quantile(S,0.95)
    95%
12.14896
```



So we have $VaR_p(S) \approx 12.14896$.

Example 5 (Profitability of a dynamic trading strategy)

An investor would like to invest his capital into the stock and a bond for 30 trading days. You are given that

- The daily log-return of riskfree bond is 0.00015 (or 0.015%)
- The daily log-return of the stock has general t-distribution with $\mu = 0.0011$, $\lambda = 0.00153$ and $\nu = 4.7$.
- The investor adopts the following dynamic trading strategy as follows:
  - Initially, the investor allocates 50% of the capital into the bond and allocates 50% of capital into the stock.
  - After that the investor will adjust the portfolio weights at the end of every trading days. The investor will invest 70% of capital into the stock if the stock return in the last trading day is positive and
  - The investor will invest 30% of capital available into the stock if the stock return in the last trading day is non-positive.

Suppose that the investor demands a return of 2% over a month, determine the probability that the trading strategy can achieve investor's goal.

😊 Solution

One can adopt Monte-Carlo simulation to estimate the probability. The algorithm is similar to earlier example except that one also needs to modify the portfolio when the new data is available.

```
n=1000000  #Number of simulation runs (sample size)
D=30 #Number of Trading days

rbond=0.00015 #Daily log-return of bond
mu=0.0011 #Daily log-return of the stock
lambda=0.00153
df=4.7
return=rep(0,n)

for (j in 1:n){
  wf=0.5 #initial portfolio
  wr=0.5
  for (i in 1:D){
    rstock=mu+lambda*rt(1,4.7) #Generate sample logreturn
    wf=wf*exp(rbond) #Update the values of the position
    wr=wr*exp(rstock)
    wtotal=wf+wr
    #Portfolio adjustment
    if(rstock>0){
      wf=0.3*wtotal
      wr=0.7*wtotal  }
    else{
```

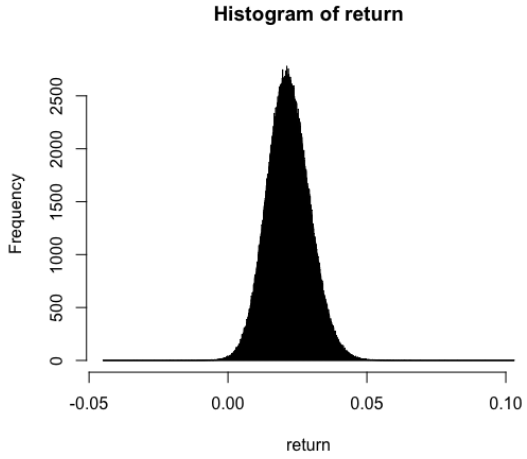
```
    wf=0.7*wtotal
    wr=0.3*wtotal } }
  return[j]=wtotal-1}
#Estimate the probability
I=ifelse(return>0.02,1,0)
estimate=mean(I)
SE=sd(I)/sqrt(n)
PE=SE/estimate
```

The result is summarized as follows:

- The probability of achieving the goal is $\hat{p} \approx 0.581655$.
- The standard error and percentage error of the estimation are $SE = 0.0004932876$ and $PE = 0.0848076\%$ respectively.

Remark of Example 5

For your information, the distribution of the monthly return is given as follows:



Statistics of monthly return

Mean: 2.174753%
Standard Deviation: 0.7630138%
Minimum: -4.496971%
25% percentile: 1.655215%
50% percentile (median): 2.153698%
75% percentile: 2.672052%
Maximum: 10.3061%

Some extra techniques for Monte Carlo simulation

Although the Monte Carlo simulation is efficient an east to use, there are two technical issues that one needs to pay attention to.

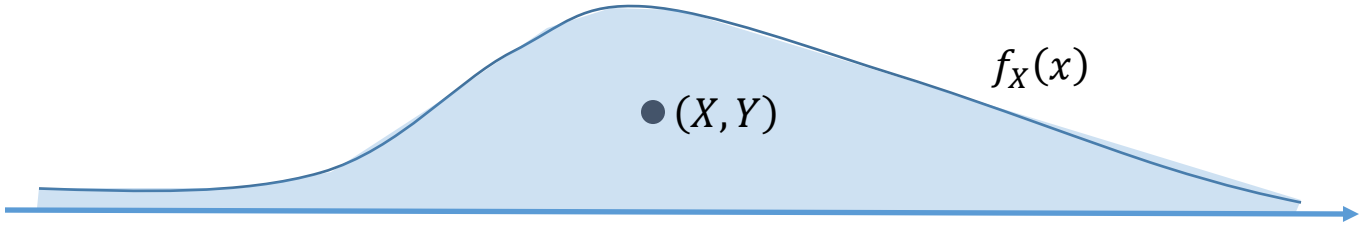
- One important step in Monte Carlo simulation is to draw a set of random sample from a given distribution $F_X(x)$.
  - If the distribution is *not* the "standard" distribution such as normal distribution, t-distribution, exponential etc., one needs to generate the random number using inverse transformation.
  - However, the method requires the explicit formula of $F_X(x) = \int_{-\infty}^{x} f_X(x)dx$ which may not be available in some cases (e.g. $f_X(x) \propto e^{-x^4}$).
- A large number of simulation run is required to achieve an estimate with good accuracy. However, it will lead to large computational time.

In this section, we will divide some methods (e.g. variance reduction technique and rejection sampling) can resolve these two issues.

Method 1: Rejection Sampling

Suppose that we would like to compute the expectation $\mathbb{E}[h(X)]$, where X is a random variable with the probability density function $f_X(x)$. This method allows us to draw the random sample from the distribution of X using another probability distribution $g_X(x)$.

Suppose that the distribution of X is visualized as follows:



As the shaded region represents the distribution of X, so one can draw a sample data of X by drawing a point *uniformly* from the region. Technically, this can be as follows:

Step 1: (x**-coordinate)** Draw a random number X from the distribution $f_X(x)$.

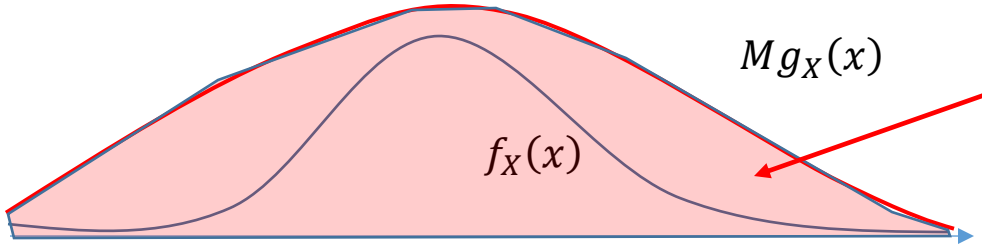
Step 2: (y**-coordinate)** Given the value of X, we draw a random number Y from uniform distribution over $[0, f_X(X)]$

One can verify that $f_{X,Y}(x, y) = f_X(x) f_{Y|X}(y) = f_X(x)\left(\frac{1}{f_X(x)}\right) = 1$. So (X, Y) can be seen as a point drawing uniformly from the region.

The rejection sampling algorithm can be summarized by the following three steps:

Step 1: We let $g_X(x)$ be another probability density function of X (Here, $g_X(x)$ is not the true distribution of X). We choose a constant $M > 0$ such that

$$Mg_X(x) \geq f_X(x) \quad for\ all\ x \in (-\infty, \infty)$$



The region below $Mg_X(x)$ can cover the entire distribution of $f_X(x)$

Step 2: We draw a random point (X^*, Y^*) from the red region as follows:

- Pick a random number X^* from the distribution of $g_X(x)$.
- Given the value of X^*, we pick a random number Y^* from uniform distribution over $[0, Mg_X(X^*)]$.

Step 3: Rejection stage

- If $Y^* > f_X(X^*)$, we reject the sample (X^*, Y^*) and redo the drawing.
- If $Y^* \leq f_X(X^*)$, we keep the x-coordinate X^*.

Next, we shall verify that the distribution of X^* has the same distribution as X.

Note that for any $x \in (-\infty, \infty)$,

$$F_{X^*}(x) = P\left(X^* \le x | Y \le f_X(X^*)\right) = \frac{P\left(X^* \le x \text{ and } Y \le f_X(X^*)\right)}{P\left(Y \le f_X(X^*)\right)}$$

$$= \frac{\int_{-\infty}^{x} g_X(z)\, P(Y \le f_X(X^*) | X^* = z) dz}{\int_{-\infty}^{\infty} g_X(z)\, \underbrace{P(Y \le f_X(X^*) | X^* = z)}_{= \int_0^{f_X(z)} \frac{1}{M g_X(z)} dy = \frac{f_X(z)}{M g_X(z)}} dz} = \frac{\frac{1}{M}\int_{-\infty}^{x} f_X(z) dz}{\frac{1}{M}\int_{-\infty}^{\infty} f_X(z) dz} = F_X(x).$$

So X^* can be viewed as a sample taken from the distribution of X.

Remark of rejection sampling

- (Choice of $g_X(x)$) One should choose $g_X(x)$ such that both the *support* of $g_X(x)$ contains the support of $f_X(x)$. That is, $g_X(x) > 0$ when $f_X(x) > 0$.
- To maximize the efficiency of the algorithm, one should choose M as small as possible such that $M = \sup_{x \in (-\infty, \infty)} \frac{f_X(x)}{g_X(x)}$. The algorithm will be very inefficient if M is too large.

Some additional technical remarks regarding rejection sampling

1. Note that Y^* is used to decide if we should accept the sample or not (see Step 2 and 3), one can modify the acceptance and rejection rule as follows:
   - Choose Y^* from uniform distribution over $[0,1]$.
   - We accept the sample if and only if $Y^* \leq \frac{f_X(X^*)}{Mg_X(X^*)}$.

   One can show that the resulting sample X^* has same distribution as X (i.e. $F_{X^*}(x) = F_X(x)$).

2. We write $f_X(x) = Cf_X^*(x)$, where C is normalizing constant such that $\int_{-\infty}^{\infty} f_X(x)dx = 1$. One can adopt the rejection algorithm by replacing $f_X(x)$ by $f_X^*(x)$. Mathematically, one can show that $F_{X^*}(x) = F_X(x)$ under this modification. Therefore, one can apply the algorithm without knowing the exact value of C (see also Example 7).

Example 6

We let X be a random variable with the following probability density function

$$f_X(x) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} x^{a-1}(1-x)^{b-1}, \qquad 0 < x < 1, \;\; a, b > 1$$

(*This distribution is known as beta distribution)

In this example, we would like to draw the random sample from this distribution using rejection sampling.

Step 1: We first choose a distribution $g_X(x)$ which its support covers the support of $f_X(x)$. Here, we will choose uniform distribution over $[0,1]$ (i.e. $g_X(x) = 1, x \in [0,1]$).

Step 2: We compute $M = \sup_{x\in[0,1]} \frac{f_X(x)}{g_X(x)} = \sup_{x\in[0,1]} f_X(x)$. This can be done by considering the first order condition:

$$\frac{d}{dx} f_X(x) = \frac{d}{dx}\left(\frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} x^{a-1}(1-x)^{b-1}\right) = 0$$

$$\Rightarrow (a-1)x^{a-2}(1-x)^{b-1} - (b-1)x^{a-1}(1-x)^{b-2} = 0 \Rightarrow x = \frac{a-1}{a+b-2}.$$

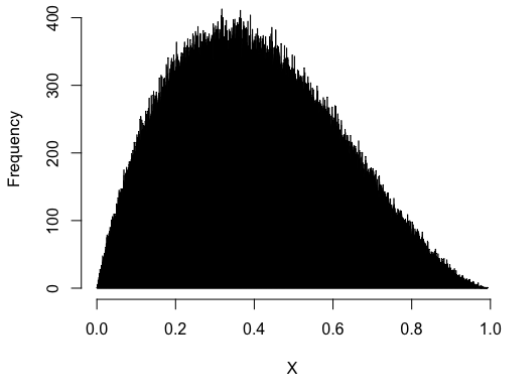
So we have $M = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \left(\frac{a-1}{a+b-2}\right)^{a-1} \left(\frac{b-1}{a+b-2}\right)^{b-1}$.

Step 3: We adopt rejection sampling algorithm. The R-code is as follows:

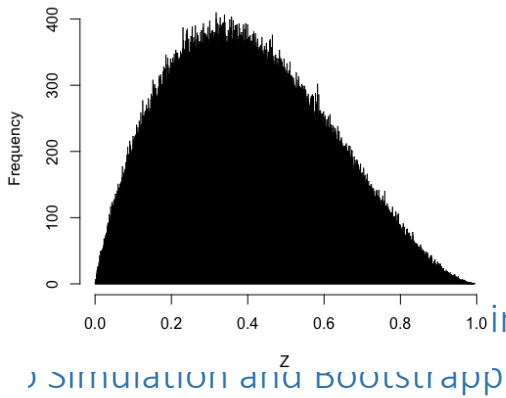
```
n=100000
a=2
b=3
X=rep(0,n)
i=1
#counter=0
M=(gamma(a+b)/(gamma(a)*gamma(b)))*(((a-1)/(a+b-2))^(a-1))*(((b-1)/(a+b-2))^(b-1))

#We used while loop because one needs to keep running the sample until we obtain certain number of sample data
while(i<=n){
 #counter=counter+1
 x=runif(1,min=0,max=1) #Draw X from the new distribution g_X(x)
 y=runif(1,min=0,max=1) #Draw Y from the uniform distribution
 threshold=(gamma(a+b)/(gamma(a)*gamma(b)))*(x^(a-1))*((1-x)^(b-1))/M
 if (y<=threshold){
  X[i]<-x
  i=i+1
  print(i) }}
```

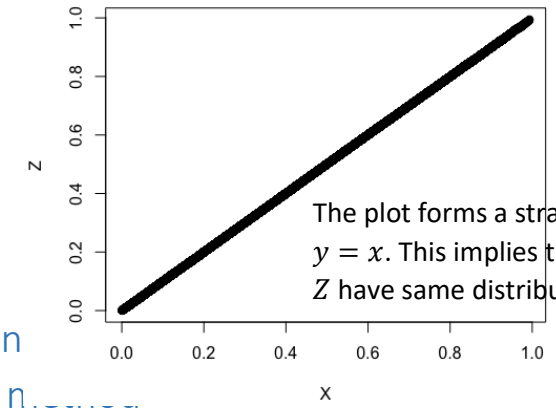
Histogram of sample data
(via rejection sampling)



Histogram of sample data
from the beta distribution



QQ-plot (X against Z)



The plot forms a straight line $y = x$. This implies that X and Z have same distribution

in Fin

) Simulation and Bootstrapping n

Example 7

We let X be a random variable which the probability density function is

$$f_X(x) = Ce^{-x^4}, \qquad x \in (-\infty, \infty)$$

where $C = \frac{1}{\int_{-\infty}^{\infty} e^{-x^4} dx}$ denotes the normalizing constant.

Suppose that we would like to estimate $\mathbb{E}[X^2]$, one can estimate it using rejection sampling as follows:

- We choose $g_X(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$ for $x \in (-\infty, \infty)$ (i.e. standard normal distribution) and take $f_X^*(x) = e^{-x^4}$.
- We consider $\frac{f_X^*(x)}{g(x)} = \sqrt{2\pi} e^{-x^4 + \frac{x^2}{2}}$. By considering the first order condition and second order condition, one can show that the ratio achieves maximum at $x = \pm\frac{1}{2}$ so that

$$M = \sup_{x \in (-\infty, \infty)} \frac{f_X^*(x)}{g(x)} = \sqrt{2\pi} e^{\frac{1}{16}}.$$

The R-code of the rejection sampling is as follows:

```
n=100000 #Target Sample Size
X=rep(0,n)
Y=rep(0,n)
i=1

M=sqrt(2*pi)*exp(1/16)

#We used while loop because one needs to keep running the sample until we obtain certain number of sample data
while(i<=n){
  x=rnorm(1,mean=0,sd=1) #Draw X from the new distribution g_X(x)
  y=runif(1,min=0,max=1) #Draw Y from the uniform distribution
  threshold=(sqrt(2*pi)/M)*exp(-(x^4)+(x^2)/2)
  if (y<=threshold){
    X[i]<-x
    Y[i]<-x^2
    i=i+1
    print(i)  }}
estimate=mean(Y)
SE=sd(Y)/sqrt(n)
PE=SE/estimate
```

The result is summarized in the following table:

	Estimate	Standard Error	Percentage Error
$\boldsymbol{N = 100000}$	0.33633356	0.001161051	0.345206%
$\boldsymbol{N = 1000000}$	0.337685	0.000368902	0.1092444%

Example 8 (Choice of distributions for rejection sampling)

We let X be a random variable with the probability density function

$$f_X(x) = \frac{x^{\alpha} e^{-x}}{x\Gamma(\alpha)}, \qquad x \geq 0,$$

where $\alpha > 1$ is a parameter and $\Gamma(\alpha) = \int_0^{\infty} t^{\alpha-1} e^{-t} dt$.

Suppose that we would like to draw random sample from this distribution through rejection sampling, which of the following distribution can we consider?

(a) $g_X(x) = e^{-x}, \quad x \geq 0$

(b) $h_X(x) = \frac{1}{(x+1)^2}, \ x \geq 0.$

(c) $p_X(x) = \frac{1}{x^3}, \ x > 1.$

□Solution

To check the feasibility, one just need to check if we can find a constant $M > 0$ such that the proposed distribution for rejection sampling can cover the *entire sampling distribution*.

It is clear that the distribution (c) is not a good candidate since it cannot cover the distribution $f_X(x)$ for $0 < x < 1$.

On the other hand, $g_X(x)$ (the distribution (a)) is not feasible also since

$$\lim_{x\to\infty} \frac{f_X(x)}{g_X(x)} = \lim_{x\to\infty} \frac{\frac{x^\alpha e^{-x}}{x\Gamma(\alpha)}}{e^{-x}} = \lim_{x\to\infty} \frac{x^{\alpha-1}}{\Gamma(\alpha)} \overset{\alpha>1}{=} +\infty.$$

This implies that for any $M > 0$, there exists x_0 such that $\frac{f_X(x)}{g_X(x)} > M \Rightarrow f_X(x) > Mg_X(x)$ for all $x > x_0$. In other words, $Mg_X(x)$ cannot cover the tail distribution of $f_X(x)$ for any $M > 0$.

Finally, the distribution (b) seems to be feasible since

$$\lim_{x\to\infty} \frac{f_X(x)}{h_X(x)} = \lim_{x\to\infty} \frac{\frac{x^\alpha e^{-x}}{x\Gamma(\alpha)}}{(1+x)^{-2}} = \frac{1}{\Gamma(\alpha)} \lim_{x\to\infty} \frac{x^{\alpha-1}(x+1)^2}{e^x} = \cdots = 0.$$

Then there exists $x_0 > 0$ such that $\frac{f_X(x)}{h_X(x)} < 1$ for $x > x_0$. On the other hand, the quotient $\frac{f_X(x)}{h_X(x)}$ has a finite maximum (say $M_0 > 0$) over the interval $[0, x_0]$. By taking $M = \max(M_0, 1)$, then we have $\frac{f_X(x)}{h_X(x)} \leq M$ or $Mh_X(x) \geq f_X(x)$.

Method 2: Importance Sampling

This method is motivated by *change of measure* technique. We let X be a random variable and $f_X(x)$ be the corresponding probability density function.

We let $g_X(x)$ be another probability density function such that both $f(x)$ and $g(x)$ have *same support*. Note that

$$\mathbb{E}_f[h(X)] = \int_{S_f} h(x) f_X(x) dx = \int_{S_g} \left(\frac{h(x) f_X(x)}{g_X(x)} \right) g_X(x) dx = \mathbb{E}_g \left[\frac{h(X) f_X(X)}{g_X(X)} \right].$$

where $S_f = \{x \in (-\infty, \infty) : f_X(x) > 0\} \subseteq \{x \in (-\infty, \infty) : g_X(x) > 0\} = S_g$.

Therefore, one can compute the expectation $\mathbb{E}_f[h(X)]$ using another probability density function $g_X(x)$.

- Similar to rejection sampling, this method is useful in computing $\mathbb{E}_f[h(X)]$ when it is difficult (or infeasible) to draw random sample from the distribution $f_X(x)$.
- On the other hand, this method can significantly improve the accuracy of the estimation under some scenarios.

Example 9

We let X be a random variable with the following probability density function

$$f_X(x) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} x^{a-1}(1-x)^{b-1}, \qquad 0 < x < 1, \;\; a, b > 1$$

We would like to evaluate $\mathbb{E}[X]$ using Monte Carlo simulation. Theoretically, one has $\mathbb{E}[X] = \frac{a}{a+b}$.

Suppose that we compute $\mathbb{E}[X]$ using importance sampling with uniform distribution over $[0,1]$ (i.e. $g_X(x) = 1, x \in (0,1)$)

According to the above discussion, it suffices to compute $\mathbb{E}[X] = \mathbb{E}_g\left[X\frac{f_X(X)}{g_X(X)}\right] = \mathbb{E}_g[Xf_X(X)]$ where X follows uniform distribution over $[0,1]$.

The corresponding R-code is presented below:

```
n=100000 #Number of simulation run

a=2 #Parameter values of original distribution
b=2

#Write a function to compute pdf of the original distribution
pdfdist<-function(X,a,b){
```

```
  D=(gamma(a+b)/(gamma(a)*gamma(b)))*(X^(a-1))*((1-X)^(b-1))
  return(D)
}

X<-runif(n,min=0,max=1) #Generate X from uniform distribution (via important sampling)

Y<-X*pdfdist(X,a,b)

estimate=mean(Y)
SE=sd(Y/sqrt(n))
PE=SE*100/estimate
```

We summarize the result in the following table. (Note: True value $= \frac{a}{a+b} = 0.5$)

	Estimate	Standard Error	Percentage Error
$\boldsymbol{N = 100000}$	0.5002703	0.00096451	0.1927968%
$\boldsymbol{N = 1000000}$	0.499789	0.0003047311	0.06097194%

As a benchmark, we also present the result which we generate random sample from the original distribution directly (by "rbeta()")

	Estimate	Standard Error	Percentage Error
$\boldsymbol{N = 100000}$	0.4996012	0.0007059551	0.1413037%
$\boldsymbol{N = 1000000}$	0.4997007	0.000223464	0.04471958%

Example 10 (Application of important sampling)

We consider a 1-year European call option on a company stock. The strike price of the option is X.

On the other hand, the current price of the stock is $S_0 = \$100$ and its future price process is governed by the following equation (known as Black-Scholes model):

$$dS_t = \mu S_t dt + \sigma S_t dW_t \Leftrightarrow S_t = S_0 e^{\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma W_t},$$

where W_t has normal distribution with mean 0 and variance t. Here, $\mu \in (-\infty, \infty), \sigma \in (0, \infty)$ denotes expected return and volatility of the stock respectively.

According to risk neutral valuation principle (or martingale pricing theory), the current price of the call option can be expressed as

$$V_0 = e^{-rT}\mathbb{E}^Q[V_T(S_T)] = e^{-rT}\mathbb{E}^Q[\max(S_T - X, 0)] \dots \dots (*)$$

where Q denotes *risk neutral probability measure* and

$$S_T = S_0 e^{\left(r - \frac{\sigma^2}{2}\right)t + \sigma W_T^Q},$$

where W_T follows normal distribution with mean 0 and variance T.

We take $r = 3\%, \sigma = 20\%,\ T = 1, N = 10000000$. We can apply classical Monte Carlo simulation and yield the following result:

	Estimate	Standard Error	Percentage Error
$\boldsymbol{X = 200}$	0.003224835	8.148452×10^{-5}	2.526781%
$\boldsymbol{X = 250}$ **(Trial)**	3.859082×10^{-5}	9.109323×10^{-6}	23.60489%
$\boldsymbol{X = 300}$ **(Trial)**	0	0	0% (!?)

The result is not satisfactory.

Mathematically, this can be explained as follows: Note that

$$\begin{aligned} V_0 &= \mathbb{E}^Q[e^{-rT}\max(S_T - X, 0)] \\ &= \underbrace{\mathbb{E}^Q[e^{-rT}\max(S_T - X, 0)\,|S_T > X]}_{(**)} P(S_T > X) \\ &\quad + \underbrace{\mathbb{E}^Q[e^{-rT}\max(S_T - X, 0)\,|S_T \le X] P(S_T \le X)}_{=0} \end{aligned}$$

Given X is large, it is rare to get a sample data with $S_T > X$ (the probability is 0.0318% for $X = 200$ and is 0.0002929% for $X = 250$). So one does not have enough sample to calculate the expectation $(**)$ accurately even though the number of simulation run is taken to be very large.

One way to resolve this problem is to adopt importance sampling and choose a distribution $g_{W_T}(x)$ such that *there is larger likelihood of picking sample data with* $S_T > X$. In this example, we shall try $g_{W_T}(x)$ to be normal distribution with mean $\mu > 0$ and variance T, where μ is chosen to be

$$X = S_0 e^{\left(r-\frac{\sigma^2}{2}\right)T+\sigma\mu} \Rightarrow \mu = \frac{\ln\left(\frac{X}{S_0}\right) - \left(r-\frac{\sigma^2}{2}\right)T}{\sigma}.$$

Here is the modified r-code for the Monte Carlo simulation

```
N=10000000 #Number of simulation runs
S0=100
X=250
sigma=0.2
r=0.03
T=1
VT=rep(0,N)
counter=0 #Count the number of sample data with ST>X
mu=(1/sigma)*(log(X/S0)-(r-(sigma^2)/2)*T)

fX=function(x){(1/sqrt(2*pi*T))*exp(-(x^2)/(2*T))} #Old dist. of WT
gX=function(x){(1/sqrt(2*pi*T))*exp(-((x-mu)^2)/(2*T))} #New dist. of WT

for (i in 1:N){
  WT=rnorm(1,mean=mu,sd=sqrt(T))
  ST=S0*exp((r-(sigma^2)/2)*T+sigma*WT)
  VT[i]=exp(-r*T)*max(ST-X, 0)*fX(WT)/gX(WT)
  if (ST>X){
```

```
  counter=counter+1
 }}
Estimate=mean(VT) #estimate
SE=sd(VT)/sqrt(N) #Standard error
PE=SE*100/Estimate #Percentage error (in percentage)
```

	Estimate	Standard Error	Percentage Error	Number of data with $S_T > X$
$\boldsymbol{X = 200}$ $\boldsymbol{(\mu = 3.4157)}$	0.003325354	1.41984×10^{-6}	0.042697%	5000858
$\boldsymbol{X = 250}$ $\boldsymbol{(\mu = 4.5315)}$	3.004098×10^{-5}	1.45971×10^{-8}	0.04859%	5000094
$\boldsymbol{X = 300}$ $\boldsymbol{(\mu = 5.4431)}$	2.730978×10^{-7}	1.456248×10^{-10}	0.053323%	5000754

One can also try a smaller simulation run ($N = 1000000$) and here is the result:

	Estimate	Standard Error	Percentage Error
$\boldsymbol{X = 200}$	0.00332851	4.492059×10^{-6}	0.13496%
$\boldsymbol{X = 250}$	2.997048×10^{-5}	4.612584×10^{-8}	0.15390%
$\boldsymbol{X = 300}$	2.732094×10^{-7}	4.603179×10^{-10}	0.16849%

Method 3 – Antithetic Variate

Given a random sample $\{X_1, X_2, \dots, X_n\}$ drawn from the distribution of X, this method aims to generate another set of sample $\{Y_1, Y_2, \dots, Y_n\}$ such that each Y_i, generated from X_i, has the same distribution as X. So we will have more sample data ($2n$ instead of n) to estimate $\mathbb{E}[h(X)]$.

How to generate the new sample $\{Y_1, Y_2, \dots, Y_n\}$?

We let $f_X(x)$ be the probability density function and suppose that

$$F_X(X_i) = U_i$$

for some $U_i \in [0,1]$. (*Note: Here, U_i is random number drawn from uniform distribution over $[0,1]$)

One can generate Y_i by setting

$$F_X(Y_i) = 1 - U_i.$$

Mathematically, one can show that X_i and Y_i has same distribution. Note that for any $x \in (-\infty, \infty)$

$$F_{Y_i}(x) = P(Y_i \le x) = P\big(F_X(Y_i) = 1 - U_i \le F_X(x)\big) = P\big(U_i \ge 1 - F_X(x)\big) = F_X(x).$$

On the other hand, one has $F_{X_i}(x) = P(X_i \le x) = P\big(F_X(X_i) = U_i \le F_X(x)\big) = F_X(x)$.

New formula for estimating $\mathbb{E}[h(X)]$

For each pair of (X_i, Y_i), we let

$$\hat{h}_i = \frac{h(X_i) + h(Y_i)}{2}.$$

Then we estimate $\mathbb{E}[h(X)]$ by

$$\underbrace{\frac{\cancel{h(X_1) + h(X_2) + \cdots + h(X_n)}}{\cancel{n}}}_{\cancel{old\ formula}} = \mathbb{E}[h(X)] \approx \underbrace{\frac{\hat{h}_1 + \hat{h}_2 + \cdots + \hat{h}_n}{n}}_{new\ formula}$$

Since $\hat{h}_1, \hat{h}_2, \dots, \hat{h}_n$ are i.i.d. and $\mathbb{E}\big[\hat{h}_i\big] = \frac{1}{2}\left(\underbrace{\mathbb{E}[h(X_i)]}_{=\mathbb{E}[h(X)]} + \underbrace{\mathbb{E}[h(Y_i)]}_{=\mathbb{E}[h(X)]}\right) = \mathbb{E}[h(X)]$, it follows from strong law of large number that $\frac{\hat{h}_1+\hat{h}_2+\cdots+\hat{h}_n}{n} \to \mathbb{E}[h(X)]$ almost surely. On the other hand, the standard error of the new estimation is seen to be

$$SE_{new} = \sqrt{\frac{Var\big(\hat{h}_i\big)}{n}} = \sqrt{\frac{1}{4n}\Big(Var\big(h(X_i)\big) + Var\big(h(Y_i)\big) + 2Cov\big(h(X_i), h(Y_i)\big)\Big)}.$$

Since $Cov\big(h(X_i), h(Y_i)\big) \leq \sqrt{Var\big(h(X_i)\big)}\sqrt{Var\big(h(Y_i)\big)}$ and $Var\big(h(X_i)\big) = Var\big(h(Y_i)\big) = Var\big(h(X)\big)$, it follows that

$$SE_{new} \leq \sqrt{\frac{1}{4n}\Big(4Var\big(h(X)\big)\Big)} = \sqrt{\frac{Var\big(h(X)\big)}{n}} = SE.$$

So the standard error is lower under the new estimation.

In addition if $h(X_i)$ and $h(Y_i)$ are negative correlated such that $Cov\big(h(X_i), h(Y_i)\big) < 0$, it follows that

$$SE_{new} < \sqrt{\frac{1}{4n}\Big(Var\big(h(X_i)\big) + Var\big(h(Y_i)\big)\Big)} = \sqrt{\frac{Var\big(h(X)\big)}{2n}}.$$

Here, $\sqrt{\frac{Var\big(h(X)\big)}{2n}}$ is seen to be the standard error of Monte Carlo simulation with random sample of size $2n$ (doubling the number of simulation run). Then the antithetic variate will be more efficient.

(*Note: It is not necessary that $h(X_i)$ and $h(Y_i)$ are always negative correlated. This depends on the choice of the function $h(x)$).

Example 11

We revisit the pricing problem as in Example 3.

Generating additional sample using Antithetic technique

We let W_T be a random number drawn from the normal distribution ($N(0,T)$) and let $F(W_T) = U$ (where $F(x)$ denotes the c.d.f. of the normal distribution $N(0,T)$).

Since the normal distribution is symmetric about $\mu = 0$, we have $F(-W_T) = 1 - F(W_T) = 1 - U$. Hence, we take $-W_T$ as the additional sample and calculate the corresponding S_T and $e^{-rT}V_T$.

The modified r-code is as follows:

```
N=10000 #Number of simulation runs
S0=100
X=110
sigma=0.2
r=0.03
T=1
VT=rep(0,N)

for (i in 1:N){
  WT=rnorm(1,mean=0,sd=sqrt(T))
```

```
  ST=S0*exp((r-(sigma^2)/2)*T+sigma*WT)
  STA=S0*exp((r-(sigma^2)/2)*T+sigma*(-WT))  #Additional sample from Antithetic variate
  VT[i]=(exp(-r*T)*max(ST-X, 0)+exp(-r*T)*max(STA-X, 0))/2
}

Estimate=mean(VT) #estimate
SE=sd(VT)/sqrt(N) #Standard error
PE=SE*100/Estimate #Percentage error (in percentage)
```

Using the same set of parameters adopted in Example 21, we obtain the following result: (*Note: The number in the bracket denotes the result under classical Monte Carlo simulation)

	Estimate	Standard Error	Percentage Error
$N = 10000$	5.352107 (5.269533)	0.06904406 (0.1079814)	1.290035% (2.049164%)
$N = 100000$	5.325369 (5.314647)	0.02142234 (0.034771417)	0.4022696% (0.6542574%)
$N = 1000000$	5.291423 (5.293702)	0.006772318 (0.01096328)	0.1279867% (0.2071004%)
$N = 10000000$	5.296571 (5.293250)	0.002144228 (0.00346183)	0.04048333% (0.0654008%)

Resampling and bootstrapping

To motivate the need of this method, we consider the following scenarios:

Scenario 1: Estimating the mean square error

We let θ be an unknown parameter and $\hat{\theta}_n$ be its point estimator. Recall that the mean square error of $\hat{\theta}_n$ is given by

$$MSE(\hat{\theta}_n) = Var(\hat{\theta}_n) + \underbrace{\left(\mathbb{E}[\hat{\theta}_n] - \theta\right)}_{Bia_\theta(\hat{\theta}_n)}^2.$$

However, it may not be easy to compute $MSE(\hat{\theta}_n)$ if the expression $\hat{\theta}_n$ is too complicated or the probability distribution of $\hat{\theta}_n$ is too complicated.

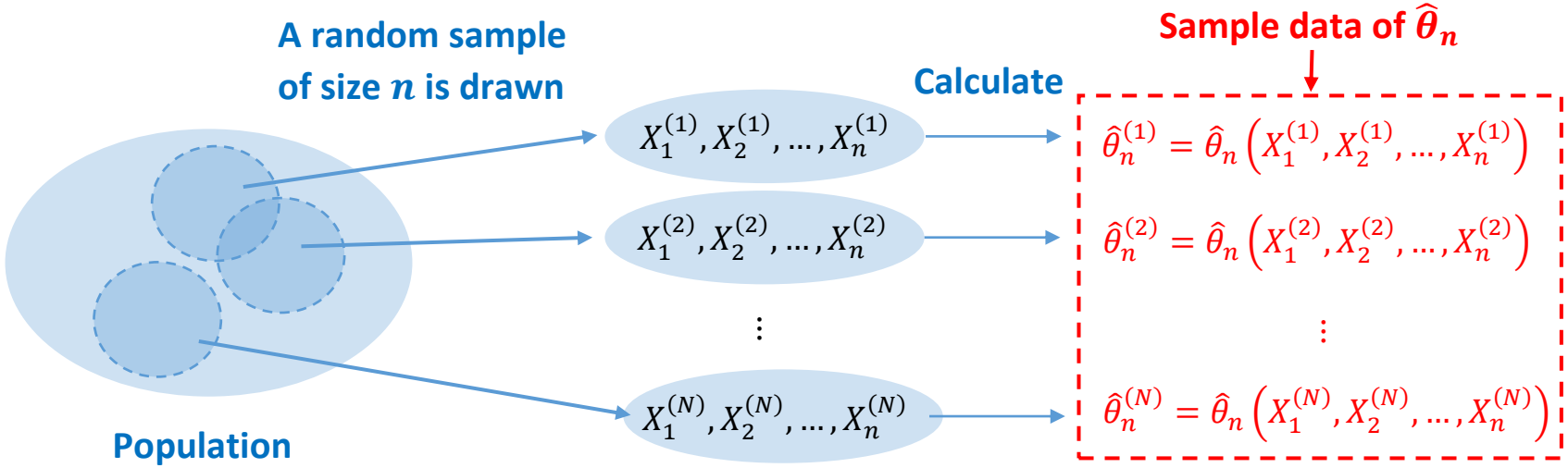
Scenario 2: Obtaining confidence interval for an unknown parameter

If the population variance σ^2 is an unknown, one needs to find the confidence interval for the population mean using the statistic $T_n = \frac{\bar{X}-\mu}{s/\sqrt{n}}$.

- If the population is normally distributed, we can still deduce that T_n has T-distribution with degree of freedom $n-1$.
- However, one cannot get the distribution of T_n when the population is NOT normally distributed.

How to overcome the difficulty?

Technically, the above two problems can be done through random resampling. More precisely, we draw a set of random sample $\left\{\hat{\theta}_n^{(1)}, \hat{\theta}_n^{(2)}, \ldots, \hat{\theta}_n^{(N)}\right\}$ from the distribution of the population as follows:



- In the context of estimating the MSE, one can estimate $Var\left(\hat{\theta}_n\right)$ through *sample variance* and estimate $\mathbb{E}\left[\hat{\theta}_n\right]$ through *sample mean*.
- In the context of finding confidence interval, the distribution of $\hat{\theta}_n$ can be estimated through *empirical distribution* of $\hat{\theta}_n$.

Does it work?

Unfortunately, this sampling strategy requires the *full information* about the true distribution of the population. Such information, clearly, is not available in real calculation. In fact, the only information available is the sample data $\{X_1, X_2, \dots, X_n\}$ drawn from the population.

In order to executing the sampling, one need to *find a substitute* for the unknown population.

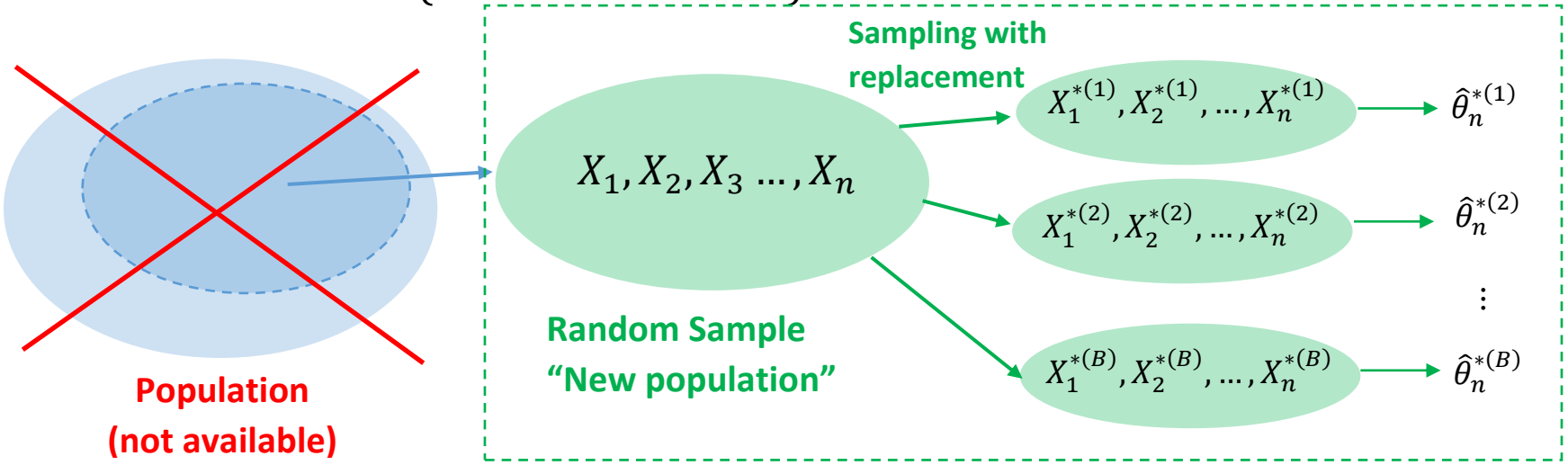
Recall that the distribution of sample data is roughly same as the distribution of original distribution when the sample size n is sufficient large, thus one can try to recover the strategy by performing the resampling from sample data $\{X_1, X_2, \dots, X_n\}$. This method is known as *bootstrapping*.

Procedure of bootstrapping

Step 1: Given the sample data $\{X_1, X_2, \dots, X_n\}$, a random sample of size n is randomly drawn *with replacement* from the sample.

Step 2: We let $\{X_1^*, X_2^*, \dots, X_n^*\}$ be the bootstrap sample obtained, we estimate $\hat{\theta}_n^{*(i)} = \hat{\theta}_n(X_1^*, \dots, X_n^*)$, which is the bootstrap sample data of $\hat{\theta}_n$.

Step 3: Repeated the procedure for sufficiently many times (say B times) and obtain a set of bootstrap sample $\left\{\hat{\theta}_n^{*(1)}, \hat{\theta}_n^{*(2)}, \ldots, \hat{\theta}_n^{*(B)}\right\}$.



Some remarks about the procedure

- Note that the sample has to be drawn with replacement, this is to ensure that $X_1^{*(i)}, \ldots, X_n^{*(i)}$ are i.i.d. and each $X_k^{*(i)}$ has the distribution $F_n(x) \approx F_X(x)$.
- In other words, bootstrapping tries to mimic the original sampling procedure by treating the random sample as the "true population".

Application of bootstrap method in statistical interference

1. Estimating the expected value, variance and bias of $\hat{\theta}_n$

We let $\hat{\theta}_n = \hat{\theta}_n(X_1, X_2, \dots, X_n)$ be a point estimator. One can estimate $\mathbb{E}[\hat{\theta}_n]$, $Var(\hat{\theta}_n)$ and $Bia(\hat{\theta}_n)$ as follows.

Given the bootstrap sample $\{\hat{\theta}_n^{*(1)}, \hat{\theta}_n^{*(2)}, \dots, \hat{\theta}_n^{*(B)}\}$, one can estimate these quantities as follows:

$$\mathbb{E}[\hat{\theta}_n] \approx \overline{\hat{\theta}_n^*} = \frac{\hat{\theta}_n^{*(1)} + \hat{\theta}_n^{*(2)} + \cdots + \hat{\theta}_n^{*(B)}}{B} \dots (*)$$

$$Var(\hat{\theta}_n) \approx \frac{1}{B-1} \sum_{i=1}^{B} \left(\hat{\theta}_n^{*(i)} - \overline{\hat{\theta}_n^*}\right)^2 \dots (**)$$

$$Bia(\hat{\theta}_n) = \mathbb{E}[\hat{\theta}_n] - \theta \approx \overline{\hat{\theta}_n^*} - \hat{\theta}_n \dots (***)$$

We replace θ by $\hat{\theta}_n$ when calculating bias. It is because the random sample is taken as the "new population", so the estimate of θ from the sample (i.e. $\hat{\theta}_n$) should represent the true value of the parameter.

Example 12 (A test on the effectiveness of bootstrap)

We consider a random sample $\{X_1, X_2, \dots, X_n\}$ drawn from lognormal distribution (with $\mu = 0$ and $\sigma = 1$, (i.e. $\ln X_i \sim N(0,1)$).

We consider point estimator of $\theta = \mathbb{E}[X]$, i.e. $\hat{\theta}_n = \frac{X_1+X_2+\cdots+X_n}{n}$. One can show theoretically that

- $\mathbb{E}[\hat{\theta}_n] = \mathbb{E}\left[\frac{X_1+X_2+\cdots+X_n}{n}\right] = \mathbb{E}[X_1] = e^{\frac{1}{2}} \approx 1.648721.$
- $Bia(\hat{\theta}_n) = \mathbb{E}[\hat{\theta}_n] - \theta = e^{\frac{1}{2}} - e^{\frac{1}{2}} = 0.$
- $SD(\hat{\theta}_n) = \sqrt{Var(\hat{\theta}_n)} = \sqrt{\frac{Var(X_i)}{n}} = \sqrt{\frac{e^2-e}{n}}.$

In this example, we would like to verify the above result using bootstrap method. To do so, we generate a set of lognormal variables (with $n = 50000$) and use it as a "dummy" random sample and estimate the quantities $\mathbb{E}[\hat{\theta}_n]$, $Bia(\hat{\theta}_n)$ and $Var(\hat{\theta}_n)$ using the bootstrap formula (*)-(***).

The R-code for this bootstrap example is shown below:

```
n=50000          #sample size
B=100000         #number of bootstrap sample
x=rlnorm(n,0,1)    #Getting sample data
theta_hat=mean(x)          # Getting θ̂_n
theta_boot=rnorm(B,0,0)
for (i in 1:B){                          #Generate a set of bootstrap sample of θ̂_n^{*(i)}
  y=sample(x,n,replace=TRUE)
theta_boot[i]=mean(y)}
mean_theta=mean(theta_boot)
bias_theta=mean(theta_boot)-theta_hat
sd_theta=sd(theta_boot)
```

The following table summarizes the estimation result from bootstrap method:

(*Note: The sample mean of sample data for bootstrap is $\mathbf{1.65512}$)

(**Note: The estimate is obtained from resampling from *true distribution*)

With $n = 50000$		*Bootstrap estimate	**Sample estimate	True value
$B = 10000$	$\mathbb{E}[\hat{\theta}_n]$	1.655107	1.648736	1.648721
	$Bia(\hat{\theta}_n)$	-1.370164×10^{-5}	1.43473×10^{-5}	0
	$SD(\hat{\theta}_n)$	0.009539141	0.0097860	0.0096652

$B = 100000$	$\mathbb{E}[\hat{\theta}_n]$	1.655122	1.648713	1.648721
	$Bia(\hat{\theta}_n)$	1.816388×10^{-6}	-8.7568×10^{-6}	0
	$SD(\hat{\theta}_n)$	0.009585167	0.0096898	0.0096652

Some comments about the result

- There are two sources of error for a bootstrap estimate: (1) Random error caused by simulation and (2) Approximation error of sample distribution.
- When the sample size is large (i.e. $n = 50000$) and the distribution of sample data can roughly approximate unknown distribution, we observe from the above table that the bootstrap estimate (especially Bia) is quite accurate when B is sufficiently large.
- We observe that the accuracy of bootstrap estimate is worse than that of sample estimate, it is because the data of sample estimate are drawn from *true distribution* so that there is no error in (2).
- When the sample size is too small, the bootstrap estimate is no longer to be accurate when we consider a very large value of B.
  For example, we take $n = 500$ and $B = 1000000$, then the bootstrap estimate becomes

With $n = 500$		*Bootstrap estimate	**Sample estimate	True value
$B = 1000000$	$\mathbb{E}[\hat{\theta}_n]$	**1.608537**	1.64859	1.648721
	$Bia(\hat{\theta}_n)$	-2.17597×10^{-5}	-0.00013173	0
	$SD(\hat{\theta}_n)$	**0.076456**	0.096541	0.0966517

Although the bootstrap estimate of bia is accurate (since the θ is replaced by $\hat{\theta}_n$ in the calculation), one can see that the estimates for expected value and standard deviation is very bad. It is simply because the sample distribution cannot approximate the unknown distribution.

- On the other hand, the bootstrap estimate of expected mean $\mathbb{E}[\hat{\theta}_n]$ is closer to the sample mean (It is 1.608558 in our example) instead of the true mean (i.e. 1.648721). This is also expected since the bootstrap sample is taken from the random sample instead of true population. However, this inconsistency can be reduced by increasing the sample size n of the random sample (however it may not be possible).

Example 13 (Application of bootstrap: Estimating MSE of maximum likelihood estimate)

In general, it is difficult to estimate the mean square error (MSE) of maximum likelihood estimate $\hat{\theta}_n$ of a parameter θ in some probability distributions.

Provided that the sample size is sufficient large, one can estimate the MSE using approximation method: Provided that $l'(\hat{\theta}_n) = 0$ and the regularity conditions are satisfied, we have

$$MSE(\hat{\theta}_n) = Var(\hat{\theta}_n) + \underbrace{\left(Bia(\hat{\theta}_n)\right)^2}_{\approx 0} \approx \frac{1}{I(\theta)},$$

where $I(\theta) = -n\mathbb{E}\left[\frac{\partial^2}{\partial\theta^2}\ln f_X(X|\theta)\right]$.

(*In R, the standard error of the MLE (generated from the command "fitdistr()") is calculated using this formula).

In this example, we shall examine whether it is possible to estimate the MSE of $\hat{\theta}_n$ using bootstrap method.

Suppose that a random variable X follows Poisson distribution with mean λ, a random sample of 10000 is drawn $\{X_1, X_2, \dots, X_{10000}\}$.

Based on the sample data, we note that

- The MLE of θ is $\hat{\theta}_n = \frac{X_1+X_2+\cdots+X_{10000}}{10000} = 3.0224$
- The MSE of the estimation, using large sample approximation, is

$$MSE(\hat{\theta}_n) \approx \frac{1}{-n\mathbb{E}\left[\frac{\partial^2}{\partial\lambda^2}\ln\left(\frac{e^{-\lambda}\lambda^X}{X!}\right)\right]} = \frac{1}{-n\mathbb{E}\left[\frac{\partial^2}{\partial\lambda^2}(-\lambda + X\ln\lambda - \ln X!)\right]}$$

$$= \frac{1}{-n\mathbb{E}\left[-\frac{X}{\lambda^2}\right]} \overset{\mathbb{E}[X]=\lambda}{=\!=} \frac{\lambda}{n} \overset{\lambda\approx 3.0224}{\approx\!\approx} 0.00030224.$$

(*Note: So the standard error is $SD = 0.01738505$)

Next, we shall compute the MSE using bootstraps. Recall that

$$MSE(\hat{\theta}_n) = Var(\hat{\theta}_n) + \left(Bia(\hat{\theta}_n)\right)^2 \approx \frac{1}{B-1}\sum_{i=1}^{B}\left(\hat{\theta}_n^{*(i)} - \overline{\hat{\theta}_n^*}\right)^2 + \left(\overline{\hat{\theta}_n^*} - \hat{\theta}_n\right)^2.$$

We adopt the following R-code for bootstrapping:

```
x<-rpois(10000,3) #For generating the data (not relevant to bootstrapping)
n=10000
B=50000
library(MASS)
library(boot)
mle_results=fitdistr(x,"Poisson") #Get the MLE of lambda (Standard Case)
Poisson_mle=function(x){
  y=fitdistr(x,"Poisson")$estimate
  return(y)}
boot_results=boot(x,Poisson_mle,B) #Perform the bootstrapp
mle_results
boot_results
```

The result is given as follows:

MLE estimate for λ (standard method via fitdistr)

	MLE $\hat{\theta}_n$	Standard error	MSE
λ	3.0224	0.01738505	0.00030224

The bootstrap estimate for standard error and MSE of $\hat{\theta}_n$ is given as follows

	Standard error	Bias	MSE
Bootstrap	0.01746354	4.5378×10^{-5}	0.0003049773

We observe that the standard error and MSE obtained from bootstrapping method is very close to that using the approximation theory.

Additional remark about Example 13 – Another use of bootstrapping method

We consider another example as follows:

A random sample is drawn from log-normal distribution with parameter $\mu = 0$ and $\sigma = 1$. Suppose that we would like to fit the data by exponential distribution with parameter λ. Our goal is to estimate the mean square error of the maximum likelihood estimate $\hat{\lambda}$.

Using similar command as in earlier example, the result is shown as follows:

MSE estimate for λ (standard method)

	MLE $\widehat{\theta}_n$	Standard error	MSE
λ	0.599394454	0.005993945	3.592738×10^{-5}

The bootstrap estimate for MSE of $\hat{\theta}_n$:

	Standard error	Bias	MSE
Bootstrap	0.008141303	0.0001382873	6.629994×10^{-5}

We observe that the mean square errors obtained by two approaches are very different. It is because the MSE obtained from the formula $\frac{1}{I(\theta)}$ is calculated based on the distribution assumed in the setup and MSE obtained from bootstrap is calculated based on the distribution of sample data (i.e. true distribution of the population). In this example, the sample data obtained is not exponentially distributed so that the MSE formula $\frac{1}{I(\theta)}$ is seens to be inaccurate.

2. Finding confidence interval of θ and executing hypothesis testing on θ

When executing the above statistical procedure, one needs to determine the probability distribution of the point estimator $\hat{\theta}_n$.

Suppose that we would like to find the confidence interval for the population mean μ, one need to determine the distribution of the sample mean $\bar{X}$ or the corresponding test statistics, i.e.

$$T = \frac{\bar{X} - \mu}{s/\sqrt{n}}.$$

However, one can only determine the distribution of T under some restrictive conditions (e.g. the population is normally distributed).

To obtain the distribution of T, one can try to obtain a random sample of T (denoted by $T^{(1)}, T^{(2)}, \ldots, T^{(B)}$) from the original distribution. When $B \to \infty$, the empiricial distribution of the random sample converges to the true distribution of T by law of large number.

Unfortunately, the original distribution is often unknown in general. If the sample size of the sample data is sufficiently large, one may seek for the approximated distribution of T using bootstrap method.

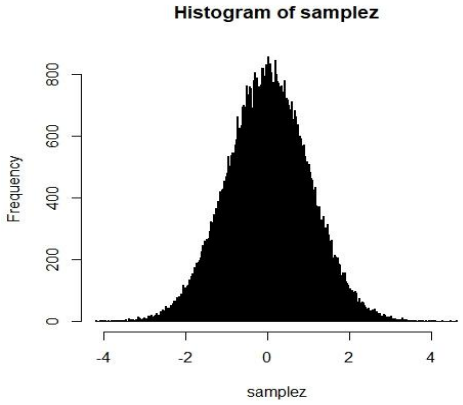
We let $\{X_1^*, X_2^* \ldots, X_n^*\}$ be a sample is drawn from the random sample and let $\bar{X}_{boot} = \frac{X_1^* + \cdots + X_n^*}{n}$ and $s_{boot}^* = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(X_i^* - \bar{X}_{boot})}$ be the bootstrap estimates of sample mean and sample standard deviation respectively. One can show that the distribution of bootstrap statistics $T^* = \frac{\bar{X}_{boot} - \bar{X}}{s_{boot}^*/\sqrt{n}}$ is approximately same as the distribution of statistics $T = \frac{\bar{X} - \mu}{s/\sqrt{n}}$ when $n \to \infty$.

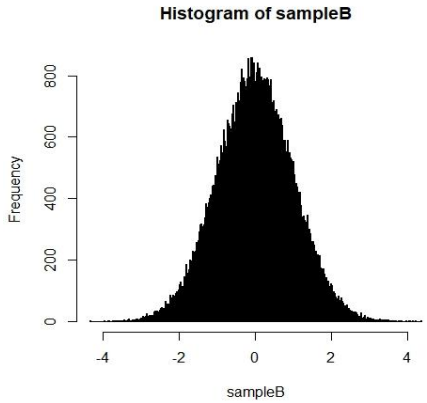
As a verification, we consider the case when the population is (1) normally distributed and (2) exponentially distributed. We compare the distribution of bootstrap statistics T^* and the true distribution of T using simulation.

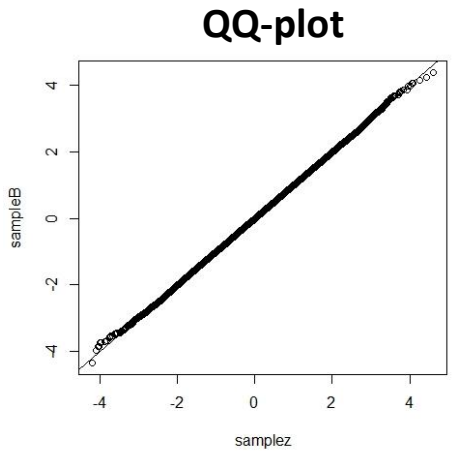
```
n=10000
B=100000
mu=2
sigma=1
samplez=rnorm(B,0,0)
sampleB=rnorm(B,0,0)
x=rnorm(n,mu,sigma)  %Generate sample data (for bootstrapping)
for (i in 1:B){
  y=rnorm(n,mu,sigma)                    %Generate sample of θ̂_n = T from true distribution
  samplez[i]=(mean(y)-mu)/(sd(y)/sqrt(n))}
for (j in 1:B){
  z=sample(x,n,replace=TRUE)             %Generate bootstrap sample of θ̂_n from sample data
  sampleB[j]=(mean(z)-mean(x))/(sd(z)/sqrt(n))}
```

The following shows the histograms of sample distribution of T and bootstrap distribution of T^* and the corresponding qqplot of bootstrap distribution against sample distribution. We observe the distributions of T and T^* are roughly the same.

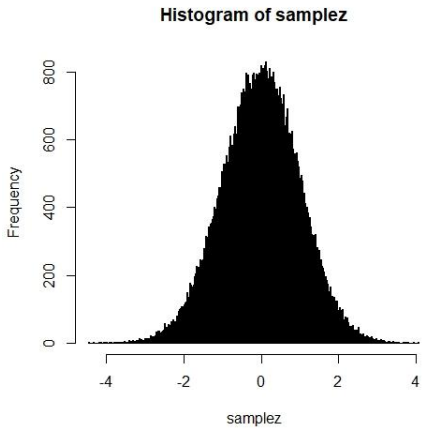
If the population is normally distributed

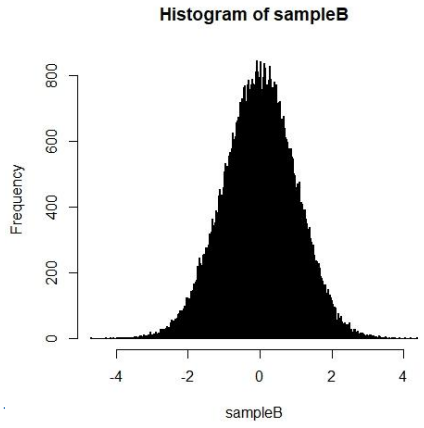


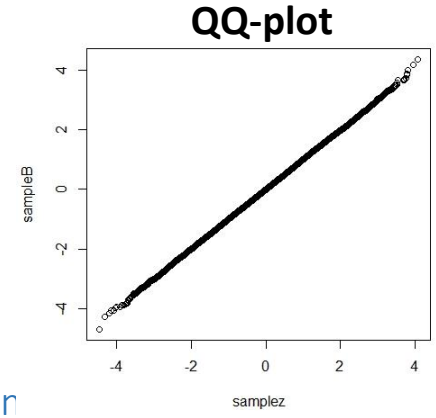




If the population is exponentially distributed







Example 14

We let μ be the average logreturn on a stock (0005.HK) In this example, we would like to demonstrate the procedure of finding confidence interval of μ and hypothesis testing on μ using bootstrap method. Here we use the data collected from 3rd Jan, 2011 to 30th Dec, 2019.

1. Finding 95% confidence interval for μ

   We first find t_L and t_U such that

$$P\left(t_L \leq T^* = \frac{\bar{X}_{boot} - \bar{X}}{s^*_{boot}/\sqrt{n}} \leq t_U\right) = 95\%,$$

   where $P(T^* \leq t_L) = 0.025$ and $P(T^* \leq t_U) = 0.975$.

   To obtain the values of t_L and t_U, we first generate a sample of $\{T_1^*, \dots, T_B^*\}$ using the sample data. Then we find the 2.5% percentile (i.e. t_L) and 97.5% percentile (i.e. t_U) of the bootstrap sample of T.

```
X0005_HK<-na.omit(X0005.HK)
x<-X0005_HK$Logreturn
n=length(x)
B=100000;
Tboot=rep(0,B)
for (i in 1:B){
  xboot=sample(x,n,replace=TRUE)
  Tboot[i]=(mean(xboot)-mean(x))/(sd(xboot)/sqrt(n))}
tL=quantile(Tboot,0.025)
tU=quantile(Tboot,0.975)
```

   We get $t_L = -1.964785$ and $t_U = 1.974351$

Hence, the 95% confidence interval of μ can be computed as follows:

$$95\% = P\left(-1.964785 \le T^* = \frac{\bar{X}_{boot} - \bar{X}}{s^*_{boot}/\sqrt{n}} \le 1.974351\right)$$

$$\approx P\left(-1.964785 \le T = \frac{\bar{X} - \mu}{s/\sqrt{n}} \le 1.974351\right)$$

$$= P\left(-1.964785 \le T = \frac{\sqrt{2213}(8.186392 \times 10^{-5} - \mu)}{0.01214441} \le 1.974351\right)$$

$$\approx P(-0.0005542 \le \mu \le 0.0007234)$$

So the 95% confidence interval of μ is

$$[-0.05542\%,\ \ 0.07234\%].$$

(*The sample mean and standard deviation are $\bar{X} = 8.186392 \times 10^{-5}$ and $s = 0.01214441$ respectively.)

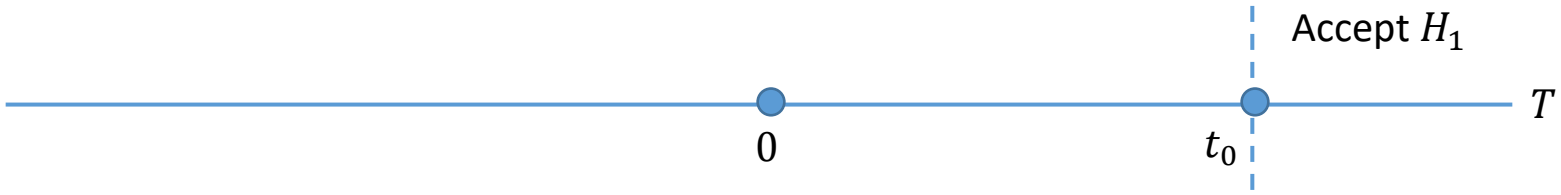
2. Hypothesis testing on μ (at significant level $\alpha = 5\%$)

   Given that the average return of 0005.HK in the past 5 years is $\bar{X} = 8.186392 \times 10^{-5}$. Suppose that we would like to check whether the average daily return of the stock is positive (i.e. $\mu > 0$).

   We first set the hypothesis as follows:

$$H_0\colon\ \mu \le 0, \qquad H_1\colon\ \mu > 0$$

Assuming that H_0 is true, one should accept H_1 only when $\bar{X}$ is much larger than $\mu = 0$ or equivalently $T = \frac{\bar{X}-\mu}{s/\sqrt{n}}$ is sufficiently large (i.e. $T > t_0$).



Given the significant level $\alpha = 5\%$, we choose t_0 such that

$$P(T > t_0) = 5\% \Leftrightarrow P(T \leq t_0) = 95\%.$$

Using the bootstrap sample, we deduce that

$$95\% = P(T \leq t_0) \approx P(T^* \leq t_0) \Rightarrow t_0 \approx 1.652879.$$

Given the sample data, the value of test statistic is

$$T = \frac{\bar{X} - \mu}{s/\sqrt{n}} = \frac{8.186392 \times 10^{-5} - 0}{0.01214441/\sqrt{2213}} \approx 0.3171076.$$

Since $T < t_0$, we accept H_0 and conclude that there is no sufficient evidence proving that $\mu > 0$.

Some comments about the bootstrap method on finding confidence interval

- In general, the method presented above can be generalized to other parameters θ: We let $\hat{\theta}_n$ and $s_n = \sqrt{Var(\hat{\theta}_n)}$ be the point estimator on θ and the standard error of $\hat{\theta}_n$ calculated from the sample respectively.
  Define $\hat{\theta}_b^*$ and s_{boot} be the bootstrap estimate and standard error obtained from a bootstrap sample. One can show that the two statistics
  $$T = \frac{\hat{\theta}_n - \theta}{s_n} \quad and \quad T^* = \frac{\hat{\theta}_b^* - \hat{\theta}_n}{s_{boot}}$$
  have roughly the same distribution when $n \to \infty$ if $\hat{\theta}_n$ is some standard statistics such as sample mean, percentiles etc..
- One possible technical difficulty is the calculation of the standard error. If $\hat{\theta}_n$ is sample mean, we calculate the standard error by $\sqrt{Var(\hat{\theta}_n)} = \sqrt{\frac{Var(X_i)}{n}}$. In general, it may not be easy (or even impossible) to obtain a formula for s_n or s_{boot}.

Parametric bootstrap

We let X be a random sample which is assumed to be certain probability distribution (e.g. normal, exponential etc.) with some unknown parameters θ.

Given the random sample $\{X_1, X_2, \dots, X_n\}$ obtained, we let $\hat{\theta}_n$ be the maximum likelihood estimate (MLE) of θ satisfying $l'(\hat{\theta}_n) = 0$. Under the regularity condition, we know that $\hat{\theta}_n$ is close to θ when the sample size n is sufficiently large.

Suppose that the model assumption is true and the true distribution of X is given by $F_X(x|\theta)$, then the distribution $F_X(x|\hat{\theta}_n)$ can approximate the unknown distribution so that the resampling can be executed using the distribution $F_X(x|\hat{\theta}_n)$.

This resampling method is known as *parametric bootstrap*. The principle of this method is very similar to the *non-parametric bootstrap* which our goal is to find a known distribution to substitute the unknown population distribution.

Example 15 (Accuracy test of parametric bootstrap)

As an example, we consider a random sample of size 10000 drawn from standard normal distribution. We let $\hat{\theta}_n = \frac{X_1+X_2+\cdots+X_n}{n}$. Our goal is to estimate the mean, bias and standard error of the estimator using parametric bootstrap.

The following is the R-code of the parametric bootstrap:

```
n=10000
B=100000
x=rnorm(n,0,1)                                    %Generate the sample data
theta_hat=mean(x)
theta_boot=rnorm(B,0,0)
mles=as.vector(fitdistr(x,"normal")$estimate)    %Obtain the MLEs based on the sample data x
mlemu=mles[[1]]
mlesigma=mles[[2]]
for (i in 1:B){                                   %parametric bootstrap
  y=rnorm(n,mlemu,mlesigma)
  theta_boot[i]=mean(y)
  print(i)}
mean_theta=mean(theta_boot)
bias_theta=mean(theta_boot)-theta_hat
sd_theta=sd(theta_boot)
```

The following table shows of numerical result of the parametric bootstrap

	Bootstrap estimate	Sample estimate	True value
$\mathbb{E}[\hat{\theta}_n]$	0.0009806934	9.9396×10^{-6}	0
$Bias(\hat{\theta}_n)$	-1.216874×10^{-5}	9.9396×10^{-6}	0
$SD(\hat{\theta}_n)$	0.009938129	0.009986785	0.01

(*Sample mean of random sample = 0.0009928621)

Provided that the sample size $n = 10000$ is fairly large, we observe that the bootstrap estimate from parametric estimate (e.g. Bias and SD) are quite accurate. However, there is a small bias on $\mathbb{E}[\hat{\theta}_n]$.

How about the case if the sample size is not too large

On the other hand, we consider the case when $n = 1000$ and examine the accuracy of the parametric bootstrap:

	Bootstrap estimate	Sample estimate	True value
$\mathbb{E}[\hat{\theta}_n]$	0.0339493	7.925279×10^{-5}	0
$Bias(\hat{\theta}_n)$	9.051211×10^{-5}	7.925279×10^{-5}	0
$SD(\hat{\theta}_n)$	0.03147072	0.03163745	0.03162278

(*Sample mean of the original sample is 0.0338588.)

We observe from the table that the bootstrap estimate can still estimate the bias and standard error accurately in this case. We can conclude that the parametric bootstrap is quite effective in estimating bias and standard error in actual application where the sample size is not too large.

Some comments about parametric bootstraps

Although parametric bootstrap provides an alternative to do the resampling when the population distribution is not available, the accuracy requires the *correct prediction* on the probability distribution of the population (Normal? Log-normal? Exponential? etc.).

In practice, it is hard to make a correct guess from sample data and the accuracy of the parametric bootstrap is questionable. Therefore, it appears that non-parametric bootstrap is more reliable.